DeepResearcher: Scaling Deep Research via Reinforcement Learning in Real-world Environments

Yuxiang Zheng[1,2,3*] **Dayuan Fu**[2,3*] **Xiangkun Hu**[2*]
Xiaojie Cai[1,3] **Lyumanshan Ye**[1,3] **Pengrui Lu**[1,3] **Pengfei Liu**[1,2,3†]
[1]Shanghai Jiao Tong University [2]Shanghai Innovation Institute
[3]Generative AI Research Lab (GAIR)
catchiz.1@sjtu.edu.cn, fdy@bupt.edu.cn, xkhu17@fudan.edu.cn
pengfei@sjtu.edu.cn

Abstract

Large Language Models (LLMs) with web search capabilities show significant potential for deep research, yet current methods—brittle prompt engineering or RAG-based reinforcement learning in controlled environments—fail to capture real-world complexities. In this paper, we introduce **DeepResearcher**, the first comprehensive framework for **end-to-end training** of LLM-based deep research agents through **scaling reinforcement learning (RL) in real-world environments** with authentic web search interactions. Unlike RAG approaches reliant on fixed corpora, DeepResearcher trains agents to navigate the noisy, dynamic open web. We implement a specialized multi-agent architecture where browsing agents extract relevant information from various webpage structures and overcoming significant technical challenges. Extensive experiments on open-domain research tasks demonstrate that DeepResearcher achieves substantial improvements of up to **28.9** points over prompt engineering-based baselines and up to **7.2** points over RAG-based RL agents. Our qualitative analysis reveals emergent **cognitive behaviors** from end-to-end RL training, such as planning, cross-validation, self-reflection for research redirection, and maintain honesty when unable to find definitive answers. Our results highlight that end-to-end training in real-world web environments is fundamental for developing robust research capabilities aligned with real-world applications. The source code for DeepResearcher is released at: https://github.com/GAIR-NLP/DeepResearcher.

1 Introduction

The emergence of Large Language Models (LLMs) has fundamentally transformed the landscape of artificial intelligence, enabling increasingly autonomous problem-solving capabilities. When equipped with external tools such as web search and code execution (Li et al., 2025c), these models can tackle complex research tasks that previously required significant human workload and expertise. Notable examples include Gemini and OpenAI Deep Research (Google, 2024; OpenAI, 2025), Grok3's DeeperSearch (xAI, 2025), and open-source projects like MetaGPT (Hong et al., 2024), OpenManus (Liang et al., 2025), and OWL agents (CAMEL-AI.org, 2025). While impressive commercial products exist, reproducible frameworks for systematically developing robust research agents remain largely elusive.

Recent advances suggest that reinforcement learning (RL) offers a promising path forward for improving LLM capabilities. Studies by Guo et al. (2025) and Team et al. (2025) demonstrate that scaling reinforcement learning for LLMs on math and coding tasks (Li et al., 2025b) substantially improves their reasoning abilities. Current open-source efforts to integrate RL with information retrieval, such as Search-R1 (Jin et al., 2025), R1-Searcher (Song et al., 2025), and ReSearch (Chen et al., 2025), have primarily focused on Retrieval-Augmented Generation (RAG) using *static*, *local* text corpora. While these approaches provide valuable insights, they **fundamentally fail to capture the dynamic, unpredictable nature of real-world web search environments**. RAG systems also fail to account for the substantial noise, variability in search quality, and the challenges of navigating diverse web content formats and structures.

In this work, we present the first comprehensive study of RL scaling for LLM agents operating with real-world web search capabilities. Our approach, **DeepResearcher**, trains agents to **interact directly with live search engines**, thereby learning to handle the inherent variability and complexity of the open web. By **training in genuine web environments rather than controlled simulations**, our system develops robust capabilities for handling

[*]Equal contribution.
[†]Corresponding author.

Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, pages 414–431
November 4-9, 2025 ©2025 Association for Computational Linguistics

the unpredictable nature of real-world information retrieval and synthesis.

DeepResearcher represents a significant departure from prompt-based and RAG-based methods. Its novelty lies in integrating several critical techniques, previously explored only in isolation, for end-to-end training in real-world web environments::

- **Scaling RL for Deep Research**: In contrast to prompt and SFT-based methods, we directly scale RL training for deep research tasks using solely outcome-based rewards.
- **Real-world Environment**: Unlike controlled RAG environments, real web search presents noisy, unstructured, and heterogeneous information sources that require sophisticated filtering and relevance assessment capabilities.
- **End-to-end Training**: We train the model end-to-end without human priors, enabling the agent to discover its own problem-solving strategies. This end-to-end approach significantly departs from human-designed workflows.
- **Addressing Implementation Challenges**: Training with real web search introduces unique challenges absent in RAG settings, including stringently managing search API rate limits, handling network latency, addressing anti-crawling mechanisms, and processing diverse webpage structures.
- **Multi-agent Framework**: Our approach employs a specialized multi-agent architecture where dedicated browsing agents extract relevant information from entire webpages—a stark contrast to RAG-based systems that simply retrieve and present pre-processed text passages.

To conclude, we make the following contributions:

- We introduce DeepResearcher, a novel RL framework specifically designed for training LLM agents in real web environments, enabling iterative reasoning and search, and synthesizing diverse web information to answer open-domain questions.
- We overcome numerous technical challenges inherent to RL scaling with real-world web search, making this the first successful implementation of RL at scale in genuine web environments.
- We conduct extensive experiments across open-domain tasks, demonstrating significant improvements over prompt-engineered baselines and RAG-based RL approaches.
- We perform detailed analysis examining emergent behaviors from DeepResearcher's end-to-end RL scaling, finding that the system can **formulate plans**, **cross-validate answers**, **reflect on its process**, and **be honest about limitations**.
- We open-source our complete training framework to the research community, fostering transparency and enabling further advancements in deep research systems.

2 Related Work

In this section, we review existing approaches to enhance large language models' (LLMs) ability to access external knowledge with search. We categorize these methods into prompt-based and training-based search agents. Furthermore, we examine the operational environments in which these methods are deployed—namely, local retrieval-augmented generation (RAG) frameworks and real-world, dynamic web search settings—and situate our approach within this broader technological and methodological landscape.

2.1 Prompt-Based Search Agents

Many current approaches rely on manually crafted workflows that specify how LLMs should interact with external knowledge sources (Wang et al., 2024a). Recent works such as OpenResearcher (Zheng et al., 2024), AirRAG (Feng et al., 2025), IterDRAG (Yue et al., 2024b), Plan*RAG (Verma et al., 2025), Search-o1 (Li et al., 2025a) and Open Deep Search (Alzubi et al., 2025) have demonstrated significant progress in search capabilities through designed workflows.

2.2 Training-Based Search Agents

Recent developments have moved beyond manually crafted prompts toward training-based approaches that enable more flexible and adaptive search behaviors.

Supervised Fine-Tuning (SFT) SFT for RAG have become an enhanced alternative to manual optimization of RAG workflows (Yu et al., 2024; Wang et al., 2024b). For example, CoRAG (Wang et al., 2024b) utilizes Monte Carlo Tree Search (MCTS) to dynamically select the best document blocks under budget constraints.

Reinforcement Learning (RL) End-to-end reinforcement learning offers a promising alternative that effectively unlocks LLMs' inherent capabilities. By late 2024, large language models

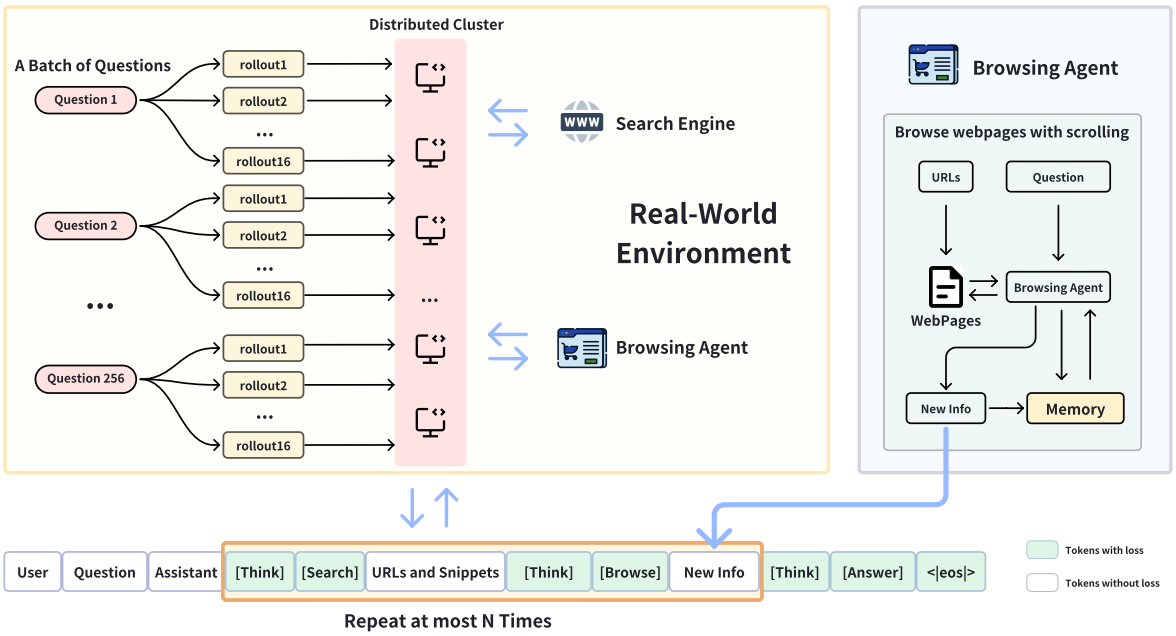


Figure 1: The trajectory of a single sample from a batch of questions processed in parallel by a distributed cluster. Each question undergoes multiple independent rollouts with distinct memory. Upper-left: Displays the batch of questions and their concurrent rollout paths. Upper-right: Shows the browsing agent retrieving web pages via URLs, processing them sequentially to incrementally extract relevant information. Bottom: Details the iterative decision-making steps, from initial query formulation and search to snippet retrieval, further reasoning, browsing, information extraction, and answer generation.

achieved remarkable breakthroughs in reasoning capability enhancement through RL (Guo et al., 2025; OpenAI, 2024; Team et al., 2025). Recent research has explored applying RL to external knowledge retrieval, with systems such as Search-R1 (Jin et al., 2025), ReSearch (Chen et al., 2025), and R1-Searcher (Song et al., 2025) abandoning manually specified cues in favor of models that autonomously develop reasoning during the retrieval process.

2.3 Training Environments

Training environments for search agents can be broadly categorized into two types:

Local RAG Environments Current mainstream local RAG frameworks (Gao et al., 2023; Yu et al., 2024) rely on pre-built fixed knowledge repositories, resulting in three critical issues: information timeliness decay, poor domain adaptability, and storage efficiency bottlenecks. While RAG-based RL approaches like Search-R1 (Jin et al., 2025), ReSearch (Chen et al., 2025), and R1-Searcher (Song et al., 2025) have made progress, their experimental validation remains primarily confined to predefined knowledge bases and similarity-based search, restricting the search space and potentially limiting generalizability to real-world applications.

Real-World Web Search Environments Web search-based methods (Schick et al., 2023; Qin et al., 2023) integrate open search engines with LLMs to access and utilize real-time information. However, search-based methods requiring external system participation are seldom trained end-to-end, with research often gravitating toward optimization through manually crafted or heuristically designed workflows (Wang et al., 2024a).

We introduce a novel approach that uniquely integrates reinforcement learning (RL) with training in genuine web environments. Unlike prior RL methods reliant on static, local corpora, our agents directly interact with live search engines. This enables them to navigate the open web's variability, developing robust capabilities for real-world information retrieval and synthesis, thereby addressing limitations of prompt-based and RAG-confined methods by learning adaptive search strategies.

3 Methodology

In this section, we describe the methodology used to train an agent capable of solving problems with web search in dynamic real-world environments.

3.1 Deep Research Trajectory

In a typical DeepResearcher's trajectory, it conducts reasoning and tool selection based on the user question and accumulated observations iteratively as illustrated in Figure 1.

Reasoning We restrict DeepResearcher to do reasoning before taking action. Each reasoning process is wrapped in a `<think>` tag following the setting in DeepSeek-R1 (Guo et al., 2025).

Web Search Tool DeepResearcher invokes the web search tool by generating a JSON-formatted request with the tool name `web_search` and the search queries as arguments. Search results are returned in a structured format comprising title, URL, and snippet for each webpage. The current implementation employs a fixed top-k (e.g., 10) value for search results retrieval. Future work could explore LLM-driven dynamic parameter optimization for enhanced search efficacy.

Web Browsing Agent The web browsing agent provides reliable, question-relevant, and incrementally updated information in to the DeepResearcher system. Specifically, the agent maintains a short-term memory repository for each query. Upon receiving a `web_browse` request, it processes the first-page segment of the URL in the request. Subsequently, the web browsing agent takes two actions based on the query, historical memory, and the newly acquired webpage content: (1) determining whether to continue reading the next URL/segment or stop and (2) appending relevant information to the short-term memory. Once the agent decides to discontinue further browsing, it compiles all newly added information from the short-term memory and returns it to the DeepResearcher system. The "specialized multi-agent architecture" mentioned earlier is an internal implementation detail of this `web_browse` tool. It is designed to effectively process information from webpages, but the primary agent's policy—which decides when and how to use this tool—is learned end-to-end without being constrained by this internal structure. Thus, the tool's architecture does not impose human priors on the agent's learned decision-making process.

Answering When the model determines it has sufficient information to answer the question, it generates a final response within <answer></answer>as the answer to return to the user.

3.2 Addressing Challenges in Dynamic Real-World Web Environments

In our open, real-world web setting, several unique challenges arise that necessitate specialized solutions. The following sections detail our strategies for managing these issues effectively.

Challenge I: High-concurrency requests at a single moment The implementation of GRPO results in a large number of sampling iterations, leading to a significant volume of search queries and webpage crawling operations (e.g., 4096), causing long delays. To resolve this issue, we created a distributed CPU server cluster with 50 nodes, specifically designed to manage the Tool requests generated during the RL rollout process. Each server is tasked with handling a portion of these requests, processing search results, and crawling webpages based on the URLs identified by the language model for further reading. It is noteworthy that the use of a 50-node cluster was determined by the hardware resources available to us; the core technical requirement is high-concurrency I/O to manage simultaneous web requests, which could also be achieved with a smaller number of more powerful servers with high-bandwidth connections.

Challenge II: Managing Web Crawling and API Limitations During the crawling phase, the system frequently encounters anti-crawl measures deployed by web servers, which may return irrelevant content or fail to respond entirely. Similarly, when interfacing with search engines or LLM APIs, restrictions such as provider rate limits (e.g. 200 per second) can arise. To mitigate these issues, we implemented a robust retry mechanism that effectively addresses exceptions encountered during API calls or webpage crawling. In addition, we introduced a caching strategy for search results: if an identical search query is made within a predetermined period (e.g., 7 days), the system retrieves the results from the cache. This approach not only reduces the API call frequency but also helps manage the associated costs, particularly for expensive services like the Google Search API.

Challenge III: Optimizing Information Extraction via a Multi-Agent Approach We employ a multi-agent framework wherein a dedicated reading agent is tasked with extracting pertinent information from crawled webpages. Given that many webpages are lengthy and may contain limited relevant content, these pages are partitioned

into smaller segments. The reading agent mimics human behavior by processing content sequentially from the first page. Under the assumption that if the initial segments of a URL predominantly contain irrelevant information, the webpage is likely unproductive and can be skipped, this method enables more efficient resource allocation and improves overall information extraction accuracy. Specifically, when handling multiple URLs, our browse tool internally utilizes several "Reading Agents" in parallel, each processing a different webpage. A "Synthesis Agent" then merges the findings into a cohesive output for the main agent.

3.3 RL Training Framework

Our approach utilizes Reinforcement Learning (RL) to train the agent. This section outlines how we employ the RL framework to train the agent and the tools used within it.

GRPO In this work, we adopt the **Group Relative Policy Optimization (GRPO)** algorithm. GRPO optimizes the current policy π_θ by leveraging a reference policy $\pi_{\theta_{\text{ref}}}$ along with a set of rollouts generated by an existing policy $\pi_{\theta_{\text{old}}}$. Specifically, given G rollouts

$$\tau = \{y_i\}_{i=1}^{G} \sim \pi_{\theta_{\text{old}}}(\cdot|x) \tag{1}$$

(with each input $x \sim D$, where D is the experience distribution), GRPO estimates the baseline using these trajectories instead of training a separate critic. The current policy is then optimized by maximizing the following objective function:

$$\mathcal{J}(\theta) = \mathbb{E}_{x\sim\mathcal{D},\{y_i\}_{i=1}^{G}\sim\pi_{\theta_{\text{old}}}(\cdot|x)} \frac{1}{G}\sum_{i=1}^{G}\left[\min\left(\frac{\pi_\theta(y_i|x)}{\pi_{\theta_{\text{old}}}(y_i|x)}A_i,\right.\right.$$
$$\left.\left.\text{clip}\left(\frac{\pi_\theta(y_i|x)}{\pi_{\theta_{\text{old}}}(y_i|x)}, 1-\epsilon, 1+\epsilon\right)A_i\right) - \beta\mathbb{D}_{\text{KL}}\left(\pi_\theta||\pi_{\theta_{\text{ref}}}\right)\right] \tag{2}$$

Masking Observations The output of the tool is an observation, not the desired result that the model is expected to produce. Therefore, we apply masking to prevent the observation from being involved in training, allowing only the model's responses to contribute to the training process.

3.4 Reward

Rewards play a crucial role during the training process and guide the agent in continuously improving its performance. This section defines the reward structure and describes how the agent's behavior is rewarded.

We employ the F1 score as our primary reward metric due to our utilization of open-domain QA datasets with short-answer ground truth. For future work involving long-form answers, more sophisticated reward may be necessary, as noted in the Deep Research system card (OpenAI, 2025). The reward is determined by the following conditions:

$$\text{reward} = \begin{cases} -1 & \text{if format is incorrect} \\ \text{F1 score} & \text{if format is correct} \end{cases}$$

- **Format Penalty**: If the format is incorrect (e.g., missing tags or structural errors), the agent receives a penalty of -1.
- **F1 Reward**: If the format is correct, the reward is based on the word-level F1 score, which measures the accuracy of the generated answer compared to the reference answer. A higher F1 score results in a higher reward.

4 Experiments

4.1 Experimental Setups

4.1.1 Training Data Curation

To ensure our models genuinely learn search strategies and to mitigate data contamination, we meticulously curated training data from existing open-domain QA benchmarks. A rigorous two-stage filtering process eliminated low-quality questions and instances where the base model could answer without search, yielding a final dataset of 80,000 examples deliberately emphasizing multi-hop scenarios (75% of the total). The complete methodology for data curation is detailed in Appendix A.

4.1.2 Model and Hyperparameters

We adopt Qwen2.5-7B-Instruct[1] (Qwen et al., 2025) as the backbone model for our training pipeline. The training is conducted using the verl framework[2]. At each training step, we sample 256 prompts, and sample 16 rollouts for each prompt. Each rollout consists of up to 10 tool calls followed by a final answer step. The training is performed with a mini-batch size of 4,096, which means one rollout stage will backprop for one time.

4.2 Evaluation and Results

4.2.1 Benchmarks

To thoroughly evaluate model performance across both in-domain (ID) and out-of-domain (OOD) set-

[1]https://huggingface.co/Qwen/Qwen2.5-7B-Instruct
[2]https://github.com/volcengine/verl

tings, we construct a diverse benchmark suite spanning a range of open-domain QA challenges. For in-domain evaluation, we include the dev sets of NQ (Kwiatkowski et al., 2019), TQ (Joshi et al., 2017), HotpotQA (Yang et al., 2018), and 2Wiki (Ho et al., 2020) as mentioned in Section A.

For out-of-domain evaluation, we introduce three datasets that differ significantly in question style and information distribution: MuSiQue (Trivedi et al., 2022), Bamboogle (Press et al., 2022), and PopQA (Mallen et al., 2022). These datasets test the model's generalization ability beyond the training domain.

To ensure a fair and balanced evaluation, we randomly sample 512 examples from the development sets of NQ, TQ, HotpotQA, 2Wiki, MuSiQue, and PopQA as well as all 125 samples from Bamboogle's development set. This sampling strategy allows us to assess model robustness across a broad range of topics and reasoning requirements.

4.2.2 Baselines

To comprehensively evaluate the effectiveness and practical utility of DeepResearcher, we compare it against the following baseline methods:

- **CoT Only**: Employs Chain-of-Thought (CoT) reasoning for answer generation without access to external reference context.
- **RAG**: Integrates CoT reasoning with retrieved reference context to guide the answer generation process.
- **Search-o1**: A multi-step reasoning baseline in which the model generates search queries or intermediate answers.[3]
- **Search-o1 + Web Search**: Extends Search-o1 by enabling open web access through real-time search APIs and URL Browse.
- **ReAct-style Agent**: A zero-shot prompting baseline where the base model is instructed to use the provided web search and browsing tools to answer questions.
- **Search-r1**: A RL method for question answering that utilizes a retriever to search Wikipedia during training and inference.
- **R1-Searcher**: Conducts Bing searches by appending `"site:en.wikipedia.org"` to queries and summarizes the top three search results. DeepResearcher differs from this approach in three key aspects: (1) DeepResearcher is trained with real-world environment; (2) DeepResearcher does not restrict the search space to a specific domain; and (3) Our method allows the model to autonomously select URLs rather than compulsorily summarizing the top three search results.
- **DeepResearcher (Local RAG)**: A direct ablation of our proposed method. This agent is trained using the exact same RL framework as DeepResearcher but is restricted to a local RAG environment instead of the live web.

[3]To ensure consistency with other results, we reimplemented search-o1 using our own prompt.

4.2.3 Evaluation Metrics

Rule-based Metrics We evaluate the performance of the model using the F1 score that aligns with the reward for training. Both ground-truth and predicted answers are normalized by converting to lowercase and removing all punctuation before computing the metrics.

Model-based Evaluation Rule-based evaluation doesn't suit long-form responses, so we adopt a model-based evaluation (MBE) approach using LLM-as-a-Judge (Zheng et al., 2023). Specifically, we prompt GPT-4o-mini (Hurst et al., 2024) to assess the model's answer against the question and ground truth answer, and label it as either "correct" or "incorrect." The MBE score is then computed as the accuracy of these judgments.(Zheng et al., 2023) The full prompt is provided in Appendix C.3.

4.2.4 Main Results

Table 1 and Table 2 present the main results of DeepResearcher and the baselines in-domain and out-of-domain, respectively. From these results, we draw the following observations:

DeepResearcher outperforms the baselines within training domains. As shown in Table 1, DeepResearcher achieves the highest performance across the four datasets when measured by the more reliable MBE metric, outperforming baselines by a substantial margin on TQ and 2Wiki. While Search-r1-base shows comparable MBE results on NQ and HotpotQA, it's important to note that Search-r1-base was specifically trained and evaluated using a local RAG system with direct access to the relevant

Method	Inference Environment	NQ		TQ		HotpotQA		2Wiki	
		F1	MBE	F1	MBE	F1	MBE	F1	MBE
Prompt Based									
CoT	Local RAG	19.8	32.0	45.6	48.2	24.4	27.9	26.4	27.3
CoT + RAG	Local RAG	42.0	59.6	68.9	75.8	37.1	43.8	24.4	24.8
Search-o1*	Local RAG	34.5	57.4	52.6	61.1	31.6	40.8	28.6	32.8
Search-o1	Web Search	32.4	55.1	58.9	69.5	33.0	42.4	30.9	37.7
ReAct-style Agent	Web Search	22.7	39.6	41.9	49.2	19.7	26.2	17.6	17.6
Training Based									
Search-r1-base	Local RAG	**45.4**	60.0	71.9	76.2	**55.9**	63.0	44.6	47.9
Search-r1-instruct	Local RAG	33.1	49.6	44.7	49.2	45.7	52.5	43.4	48.8
R1-Searcher	Web Search	35.4	52.3	73.1	79.1	44.8	53.1	59.4	65.8
DeepResearcher (Local RAG)	Local RAG	29.5	46.3	51.9	55.5	29.4	35.4	26.3	27.5
DeepResearcher	Web Search	39.6	**61.9**	**78.4**	**85.0**	52.8	**64.3**	**59.7**	**66.6**

Table 1: In-domain results on four datasets (NQ, TQ, HotpotQA, 2Wiki), evaluated by F1 and MBE metrics. DeepResearcher outperforms all baseline methods in MBE and shows competitive performance in F1, particularly excelling on TQ and 2Wiki. It is worth noting that Search-r1-base was trained and evaluated in a local RAG environment with direct access to the relevant Wikipedia corpus, while DeepResearcher must navigate the entire Internet to find information, achieving excellent results despite facing a more realistic and challenging scenario.

Wikipedia corpus. In contrast, DeepResearcher must navigate the entire Internet to find relevant information, representing a more realistic and significantly more challenging scenario even though the answers ultimately come from Wikipedia.

DeepResearcher demonstrates exceptional generalization to novel domains. As revealed in Table 2, DeepResearcher consistently outperforms all other baselines across three OOD datasets. This indicates that the model successfully learns generalizable skills for reasoning, searching, and synthesizing information from different sources through RL scaling, rather than merely adapting to specific training distributions.

Importance of Real-World Environment in Training The most direct evidence comes from our ablation study comparing DeepResearcher with its counterpart trained in a local RAG environment. The results show a dramatic drop in performance for "DeepResearcher (Local RAG)" across all datasets, which empirically validates our central thesis: the noisy and dynamic nature of the live web is a necessary training ground for fostering generalizable and robust research capabilities. This advantage is further exemplified on benchmarks like Bamboogle, which requires knowledge beyond Wikipedia's coverage. On this dataset, not only does DeepResearcher significantly outperform local RAG-based methods, but it also surpasses R1-Searcher even when the latter is granted web access at inference time. These results collectively demonstrate that end-to-end training in a real-world environment develops robust information retrieval and synthesis skills that cannot be replicated in controlled, static settings.

5 Analysis

5.1 Training Dynamics

- **Performance gradually scaling with reinforcement learning:** Figure 2 (a) present the evaluation of F1 scores, across different training steps. The F1 score 0.375, and gradually increases to around 0.55 demonstrating a consistent upward trend. This result indicates the progressive improvement of the model's performance in reinforcement learning.

- **Training leads to increased reasoning steps in hard question:** Figure 2 (b) illustrates the average number of turns required for different reasoning hops. The general trend indicates that as the training progresses, the required number of tool calls also increases across different difficulty levels. Unlike the other three settings, the 4-hop setting continues to exhibit an increasing trend even after 34 steps. This suggests that the model is still learning to retrieve more information when dealing with more difficult questions.

- **Continuous learning makes long response without saturation:** Figure 2 (c) presents the length of responses for different reasoning hops. The response lengths also increase with reasoning complexity. However, all four settings show a sustained upward trend, indicating that the model continues to expand

Method	Inference Environment	Musique		Bamboogle		PopQA	
		F1	MBE	F1	MBE	F1	MBE
Prompt Based							
CoT	Local RAG	8.5	7.4	22.1	21.6	17.0	15.0
CoT + RAG	Local RAG	10.0	10.0	25.4	27.2	46.9	48.8
Search-o1*	Local RAG	16.8	21.3	35.8	38.4	36.9	42.4
Search-o1	Web Search	14.7	19.7	46.6	53.6	38.3	43.4
ReAct-style Agent	Web Search	8.9	10.0	34.4	36.8	19.1	20.5
Training Based							
Search-r1-base	Local RAG	26.7	27.5	56.5	57.6	43.2	47.0
Search-r1-instruct	Local RAG	26.5	28.3	45.0	47.2	43.0	44.5
R1-Searcher	Web Search	22.8	25.6	64.8	65.6	42.7	43.4
DeepResearcher (Local RAG)	Local RAG	12.7	12.5	42.7	46.4	23.2	23.4
DeepResearcher	Web Search	**27.1**	**29.3**	**71.0**	**72.8**	**48.5**	**52.7**

Table 2: This table shows the performance of different methods on three out-of-domain datasets (Musique, Bamboogle, PopQA), evaluated by F1 and MBE metrics. DeepResearcher leads in both F1 and MBE on all datasets, demonstrating strong generalization capabilities compared to other methods. Notably, unlike the other datasets, Bamboogle's corpus is not entirely derived from Wikipedia pages.

its reasoning processes during training. This further supports the idea that the model adapts to increasingly complex queries by generating more detailed outputs like double-check, refinement, planning, etc.

5.2 Case Study

Figures 3 and 4 present four cases illustrating the model's behavior after reinforcement learning. From these examples, we identify several key behavioral patterns:

- **Behavior I: Planning when addressing multi-hop questions:** As demonstrated on the left side of Figure 3, DeepResearcher is capable of making plans and dynamically adjusting it throughout the reasoning process. Notably, the model can merge steps when appropriate, indicating that planning abilities emerge naturally without the necessity of SFT on explicit planning data (Yue et al., 2024a).

- **Behavior II: Cross-validation before finalizing its answers:** As observed on the right side of Figure 3, DeepResearcher identifies the correct answer during its first tool call. However, rather than immediately committing to this result, it proceeds to verify its accuracy through subsequent steps. This cautious approach enhances the reliability of model's responses, ensuring greater robustness in final predictions.

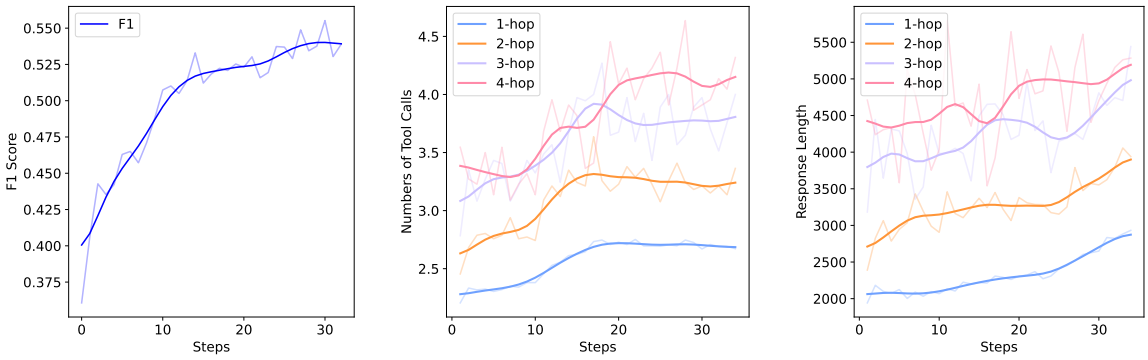
- **Behavior III: Reflection when observations deviate from expectations:** The left side of Figure 4 illustrates the model's ability to reflect on its search process. When the retrieved information does not fully align with the question, DeepResearcher recognizes this discrepancy based on environmental feedback and refines its search query in subsequent tool calls. This reflective capability is essential for preventing the model from getting stuck (Fu et al., 2025) in reasoning, enabling it to enhance overall problem-solving efficiency.

- **Behavior IV: Honesty by acknowledging its limitations:** A reliable model should minimize hallucinations and provide honest responses when it lacks the necessary knowledge (Yang et al., 2024). We observe that DeepResearcher is capable of recognizing when it has not found the correct answer and appropriately declines to provide a response. This behavior is beneficial, however, current question-answering evaluation metrics do not yet account for this aspect of model reliability.

6 Conclusion

In conclusion, we presents DeepResearcher, a groundbreaking approach for scaling reinforcement learning in LLMs to operate effectively in real-world web search environments. Unlike approaches dependent on static knowledge bases, DeepResearcher trains agents to interact with live search engines, allowing them to navigate the inherent complexity and variability of the open web. This direct engagement with dynamic search environments leads to substantial improvements in task completion and deep research capabilities.

Through an end-to-end training methodology, DeepResearcher addresses real-world challenges like network latency while enabling agents to autonomously develop robust problem-solving strate-



(a) (b) (c)

Figure 2: Training dynamics of F1, turns and response length. In this figure, we find the performance gradually scaling with reinforcement learning. The numbers of tool calls and responses also increase.

gies and cultivates cognitive behaviors such as planning, reflection, and cross-validation through its multi-agent architecture. The success of DeepResearcher represents a significant milestone for LLM agents, showcasing how scaling reinforcement learning in real-world environments can unlock superior research performance and pave the way for more adaptive systems capable of tackling complex, open-domain problems.

Limitations

While DeepResearcher demonstrates significant advancements, this study has certain limitations. Firstly, our experiments were conducted using a 7B parameter model (Qwen2.5-7B-Instruct). Although this model size yielded substantial improvements, we have not yet explored the potential performance gains or emergent capabilities that might arise from applying the DeepResearcher framework to significantly larger language models. Future work could investigate the scalability of our approach with more powerful base models.

Secondly, the reward mechanism employed in this study, while effective for the open-domain QA tasks with short, factual answers (relying on F1 scores and a format penalty), may not adequately address the complexities of more open-ended deep research inquiries. Such inquiries often involve ill-defined problem spaces, require extensive synthesis of diverse information, and may result in long-form, nuanced outputs where traditional metrics like F1 are less applicable. Consequently, a critical area for future exploration is the development of training methodologies and reward structures specifically tailored to deep research on these more open questions.

Ethical Considerations

The advanced information retrieval and reasoning capabilities of DeepResearcher, while powerful, could potentially be misused by malicious actors for tasks such as infringing upon individual privacy by accessing sensitive information without consent. We emphasize that the DeepResearcher framework is intended for beneficial research, and developers must ensure its responsible and ethical application, adhering to privacy and legal standards.

Acknowledgments

The authors would like to thank the anonymous reviewers for their suggestions and feedback on the work. This work was partially funded by the National Natural Science Foundation of China (62476168), National High Technology Research and Development Program of China (2015AA015408), Shanghai Science and Technology Development Funds (14ZR1403200). This project is supported by SJTU SEIEE - ByteDance Large Language Model Joint Laboratory.

References

Salaheddin Alzubi, Creston Brooks, Purva Chiniya, Edoardo Contente, Chiara von Gerlach, Lucas Irwin, Yihan Jiang, Arda Kaz, Windsor Nguyen, Sewoong Oh, and 1 others. 2025. Open deep search: Democratizing search with open-source reasoning agents. *arXiv preprint arXiv:2503.20201*.

CAMEL-AI.org. 2025. Owl: Optimized workforce learning for general multi-agent assistance in real-world task automation. https://github.com/camel-ai/owl. Accessed: 2025-03-07.

Mingyang Chen, Tianpeng Li, Haoze Sun, Yijie Zhou, Chenzheng Zhu, Haofen Wang, Jeff Z. Pan, Wen

Zhang, Huajun Chen, Fan Yang, Zenan Zhou, and Weipeng Chen. 2025. Research: Learning to reason with search for llms via reinforcement learning. *Preprint*, arXiv:2503.19470.

Wenfeng Feng, Chuzhan Hao, Yuewei Zhang, Jingyi Song, and Hao Wang. 2025. Airrag: Activating intrinsic reasoning for retrieval augmented generation via tree-based search. *arXiv preprint arXiv:2501.10053*.

Dayuan Fu, Keqing He, Yejie Wang, Wentao Hong, Zhuoma Gongque, Weihao Zeng, Wei Wang, Jingang Wang, Xunliang Cai, and Weiran Xu. 2025. Agentrefine: Enhancing agent generalization through refinement tuning. *arXiv preprint arXiv:2501.01702*.

Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Haofen Wang, and Haofen Wang. 2023. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2.

Google. 2024. Gemini deep research.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, and 1 others. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*.

Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. 2020. Constructing a multi-hop QA dataset for comprehensive evaluation of reasoning steps. In *Proceedings of the 28th International Conference on Computational Linguistics*, pages 6609–6625, Barcelona, Spain (Online). International Committee on Computational Linguistics.

Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2024. MetaGPT: Meta programming for a multi-agent collaborative framework. In *The Twelfth International Conference on Learning Representations*.

Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, and 1 others. 2024. Gpt-4o system card. *arXiv preprint arXiv:2410.21276*.

Bowen Jin, Hansi Zeng, Zhenrui Yue, Dong Wang, Hamed Zamani, and Jiawei Han. 2025. Search-r1: Training llms to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516*.

Mandar Joshi, Eunsol Choi, Daniel Weld, and Luke Zettlemoyer. 2017. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1601–1611, Vancouver, Canada. Association for Computational Linguistics.

Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, Ming-Wei Chang, Andrew M. Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. 2019. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:452–466.

Xiaoxi Li, Guanting Dong, Jiajie Jin, Yuyao Zhang, Yujia Zhou, Yutao Zhu, Peitian Zhang, and Zhicheng Dou. 2025a. Search-o1: Agentic search-enhanced large reasoning models. *arXiv preprint arXiv:2501.05366*.

Xuefeng Li, Haoyang Zou, and Pengfei Liu. 2025b. Limr: Less is more for rl scaling. *Preprint*, arXiv:2502.11886.

Xuefeng Li, Haoyang Zou, and Pengfei Liu. 2025c. Torl: Scaling tool-integrated rl. *Preprint*, arXiv:2503.23383.

Xinbin Liang, Jinyu Xiang, Zhaoyang Yu, Jiayi Zhang, and Sirui Hong. 2025. Openmanus: An open-source framework for building general ai agents. `https://github.com/mannaandpoem/OpenManus`.

Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Hannaneh Hajishirzi, and Daniel Khashabi. 2022. When not to trust language models: Investigating effectiveness and limitations of parametric and non-parametric memories. *arXiv preprint*.

OpenAI. 2024. Learning to reason with llms, september 2024.

OpenAI. 2025. Deep research system card. Technical report, OpenAI.

Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A Smith, and Mike Lewis. 2022. Measuring and narrowing the compositionality gap in language models. *arXiv preprint arXiv:2210.03350*.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, and 1 others. 2023. Toolllm: Facilitating large language models to master 16000+ real-world apis. *arXiv preprint arXiv:2307.16789*.

Qwen, :, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, and 25 others. 2025. Qwen2.5 technical report. *Preprint*, arXiv:2412.15115.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36:68539–68551.

Huatong Song, Jinhao Jiang, Yingqian Min, Jie Chen, Zhipeng Chen, Wayne Xin Zhao, Lei Fang, and Ji-Rong Wen. 2025. R1-searcher: Incentivizing the search capability in llms via reinforcement learning. *arXiv preprint arXiv:2503.05592*.

Kimi Team, Angang Du, Bofei Gao, Bowei Xing, Changjiu Jiang, Cheng Chen, Cheng Li, Chenjun Xiao, Chenzhuang Du, Chonghua Liao, and 1 others. 2025. Kimi k1.5: Scaling reinforcement learning with llms. *arXiv preprint arXiv:2501.12599*.

Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2022. MuSiQue: Multi-hop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*.

Prakhar Verma, Sukruta Prakash Midigeshi, Gaurav Sinha, Arno Solin, Nagarajan Natarajan, and Amit Sharma. 2025. Plan*rag: Efficient test-time planning for retrieval augmented generation. *Preprint*, arXiv:2410.20753.

Xiaohua Wang, Zhenghua Wang, Xuan Gao, Feiran Zhang, Yixin Wu, Zhibo Xu, Tianyuan Shi, Zhengyuan Wang, Shizheng Li, Qi Qian, and 1 others. 2024a. Searching for best practices in retrieval-augmented generation. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 17716–17736.

Ziting Wang, Haitao Yuan, Wei Dong, Gao Cong, and Feifei Li. 2024b. Corag: A cost-constrained retrieval optimization system for retrieval-augmented generation. *arXiv preprint arXiv:2411.00744*.

xAI. 2025. Grok 3.

Yuqing Yang, Ethan Chern, Xipeng Qiu, Graham Neubig, and Pengfei Liu. 2024. Alignment for honesty. *Advances in Neural Information Processing Systems*, 37:63565–63598.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Tian Yu, Shaolei Zhang, and Yang Feng. 2024. Auto-rag: Autonomous retrieval-augmented generation for large language models. *arXiv preprint arXiv:2411.19443*.

Murong Yue, Wenlin Yao, Haitao Mi, Dian Yu, Ziyu Yao, and Dong Yu. 2024a. Dots: Learning to reason dynamically in llms via optimal reasoning trajectories search. *arXiv preprint arXiv:2410.03864*.

Zhenrui Yue, Honglei Zhuang, Aijun Bai, Kai Hui, Rolf Jagerman, Hansi Zeng, Zhen Qin, Dong Wang, Xuanhui Wang, and Michael Bendersky. 2024b. Inference scaling for long-context retrieval augmented generation. *arXiv preprint arXiv:2410.04343*.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*.

Yuxiang Zheng, Shichao Sun, Lin Qiu, Dongyu Ru, Cheng Jiayang, Xuefeng Li, Jifan Lin, Binjie Wang, Yun Luo, Renjie Pan, Yang Xu, Qingkai Min, Zizhao Zhang, Yiwen Wang, Wenjie Li, and Pengfei Liu. 2024. OpenResearcher: Unleashing AI for accelerated scientific research. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 209–218, Miami, Florida, USA. Association for Computational Linguistics.

A Beyond Memorization: Curating Search-Dependent Training Data

A.1 Leveraging Open Domain QA Data

Despite the growing interest in deep research capabilities for LLM agents, there currently exists no open-source training dataset specifically designed for this purpose. To address this gap, we leverage existing open-domain question-answering datasets, which contain single-hop to multi-hop questions that inherently require online search to find accurate answers.

Our training corpus comprises a diverse collection of QA datasets that require varying degrees of retrieval complexity. Specifically, we utilize NaturalQuestions (NQ) (Kwiatkowski et al., 2019) and TriviaQA (TQ) (Joshi et al., 2017) for single-hop scenarios, where answers can typically be found within a single web document. For more complex multi-hop scenarios, which require integrating information across multiple sources, we incorporate examples from HotpotQA (Yang et al., 2018) and 2WikiMultiHopQA (2Wiki) (Ho et al., 2020), both of which were specifically designed to evaluate multi-step reasoning capabilities.

A.2 The Issue of Data Contamination

For training models that genuinely learn to leverage web search tools—rather than simply recalling memorized information—it is critical to address the problem of data contamination. Large language models have been pretrained on vast internet corpora, which likely include many of the QA pairs in standard benchmarks. Without proper contamination detection, the model might appear to successfully complete research tasks while actually using its parametric knowledge, defeating the purpose of learning web search strategies.

This contamination issue is particularly problematic in the context of our work, as it could lead to:

- Models that falsely appear to benefit from web search when actually using memorized knowledge.
- Failure to develop genuine search strategies when deployed on truly novel questions.
- Inability to generalize to real-world research scenarios where answers cannot be found in the model's training data.

A.3 Data Cleaning and Contamination Detection

To ensure the integrity of our training process, we implemented a comprehensive two-stage filtering methodology:

Low-Quality Question Filtering We exclude questions that could yield unreliable or problematic search results. Specifically, we eliminate: 1) Time-sensitive questions (e.g., "Who is the current CEO of Apple?"); 2) Highly subjective queries (e.g., "What is the best smartphone?"); and 3) Potentially harmful or policy-violating content. This filtering was implemented using DeepSeek-R1 (Guo et al., 2025) with a carefully designed evaluation prompt to systematically identify and mark problematic questions.

Contamination Detection To ensure the model genuinely learns to use search tools rather than memorizing answers, we employed a robust contamination detection procedure. For each candidate question, we randomly sample 10 responses from the base model we will use in training, and check if any response contains the ground truth answer (i.e., pass@10). Questions where the model demonstrated prior knowledge (by producing the correct answer without search) were excluded from the training set. This contamination screening is critical for preventing the model from developing a false reliance on parametric knowledge when search-based knowledge is required.

The prompts used for data cleaning and contamination detection are listed in Appendix C.1. After applying these quality control measures, we constructed a final training dataset of 80,000 examples with a distribution ratio of 1:1:3:3 for NQ:TQ:HotpotQA:2Wiki. This proportion deliberately emphasizes multi-hop scenarios (75% of examples), as these better reflect the complex information-seeking behaviors required for deep research questions.

B Case Study Example

This Section provides supplementary visual examples to illustrate distinct behavioral characteristics of the DeepResearcher during its reasoning processes. These case studies serve to concretize the model's capabilities in dynamic planning, cross-validation, reflection, and honest self-assessment.

Planning

Question: *What is the name of the famous bridge in the birthplace of Gloria in D Major's composer?*
Ground Truth: Rialto Bridge

Response Trajectory from DeepResearcher:

<think>
To answer this question, I need to figure out who Gloria in D Major's composer was and their birthplace. Then I can find the famous bridge in that city.

Step 1: Identify the composer of Gloria in D Major.
Step 2: Find the place of birth of that composer.
Step 3: Look for a famous bridge in that city.
</think>
<search>
composer of Gloria in D Major
birthplace of the composer of Gloria in D Major
</search>

Make a plan first

<tool_response>
{
'title': 'Gloria (Vivaldi) - Wikipedia',
'snippets': ' Antonio Vivaldi wrote at least three Gloria compositions, settings of the hymn Gloria in excelsis Deo, with words probably dating back to the 4th century.',
'url':'https://en.wikipedia.org/wiki/Gloria_(Vivaldi)'
}
{
'title': 'Antonio Vivaldi: Gloria in D Major, RV 589 - Boston Baroque',
'snippets': ' The Gloria, part of that Turin collection, received its twentieth-century premiere in 1930 and has remained the most popular of all Vivaldi's vocal works ever ...',
'url':'https://baroque.boston/vivaldi-gloria'
}
...(more results are omitted to save space)
</tool_response>

Step 1

<think>
From the search results, **I can see that Antonio Vivaldi is the composer of Gloria in D Major** and **he was born in Venice**. Now I need to find the famous bridge in Venice.
</think>
<search>
famous bridge in Venice
</search>

Step 2

<tool_response>
{
'title': 'THE 15 BEST Venice Bridges to Visit (2025) - Tripadvisor',
'snippets': 'Ponte della Paglia is a bridge that bridges the Rio di Palazzo canal on the border of the Sestiere San Marco and Cast',
'url':'https://www.tripadvisor.com/Attractions-g187870-Activities-c47-t5-Venice_Veneto.html'
}
{
'title': 'Rialto Bridge - Wikipedia',
'snippets': 'The Rialto Bridge is the oldest of the four bridges spanning the Grand Canal in Venice, Italy. Connecting the sestieri (districts) of San Marco and San Polo ...',
'url':'https://en.wikipedia.org/wiki/Rialto_Bridge'
}
...(more results are omitted to save space)
</tool_response>
<think>
The Rialto Bridge is the oldest and most famous bridge in Venice. I can now answer the question.
</think>

Step 3

<answer>
Rialto Bridge
</answer>

Cross Validation

Question: *Which filmmaker is considered a pioneer, Herschell Gordon Lewis or Leah Meyerhoff?*
Ground Truth: *Herschell Gordon Lewis was a pioneer*

Response Trajectory from DeepResearcher:

<think>
To answer this question, I need to investigate the contributions of both Herschell Gordon Lewis and Leah Meyerhoff to filmmaking. Specifically, I need to understand if either of them is considered a pioneer. First, I will perform a web search to find out more about the filmmakers and what they are known for.
</think>
<search>
Herschell Gordon Lewis filmmaker pioneer
Leah Meyerhoff filmmaker pioneer
</search>
<tool_response>
{
'title': 'Herschell Gordon Lewis - Wikipedia',
'snippets': 'Herschell Gordon Lewis (June 15, 1926 September 26, 2016) was an American filmmaker, best known for creating the 'splatter' subgenre of horror films.',
'url':'https://en.wikipedia.org/wiki/Herschell_Gordon_Lewis'
}
...(more results are omitted to save space)
</tool_response>
<think>
The web search results show that Herschell Gordon Lewis is frequently referred to as a pioneer in the genre of horror films, often being called the "Godfather of Gore" and "Pioneer in the arena of gore or splatter flicks." On the other hand, Leah Meyerhoff is described as an award-winning independent filmmaker, particularly known for her narrative feature film "I Believe In Unicorns". There is no mention of her being a pioneer in the filmmaking industry.

Based on the information retrieved, it seems that Herschell Gordon Lewis is the filmmaker considered a pioneer.

The next step is to narrow down the exact reason why he is considered a pioneer. I will conduct another web search with more specific keywords to **pinpoint the characteristics of his work that set him apart as a pioneer**.
</think>
<search>
Herschell Gordon Lewis why considered a pioneer
Herschell Gordon Lewis pioneer features
</search>

Validate result from different sources

<tool_response>
{
'title': 'Herschell Gordon Lewis, a Pioneer of Gore Cinema, Dies at 90',
'snippets': 'Mr. Lewis set the bar for low-budget horror in movies like "Blood Feast" that went on to inspire filmmakers like Wes Craven and Quentin ...',
'url':'https://www.nytimes.com/2016/09/28/movies/herschell-gordon-lewis-a-pioneer-of-gore-cinema-dies-at-90.html'
}
...(more results are omitted to save space)
</tool_response>
<think>
From the refined search, it is clear that Herschell Gordon Lewis is considered a pioneer due to his pioneering the "splatter" subgenre of horror films. His films introduced the extreme visuals and violent content that set them apart, even though they were often considered an "insult to the art of filmmaking". Therefore, Herschell Gordon Lewis is the filmmaker considered a pioneer.
</think>
<answer>
Herschell Gordon Lewis
</answer>

Figure 3: Planning and Cross Validation Behaviors. **Left**: DeepResearcher is capable of making plans and dynamically adjusting it throughout the reasoning process. Notably, the model can merge steps when appropriate. **Right**: DeepResearcher identifies the correct answer during its first tool call. However, rather than immediately committing to this result, it proceeds to verify its accuracy through subsequent steps. This rigorous behavior enhances the reliability of the model's responses, ensuring greater robustness in its final answers. Note: In the actual model, the tool call and tool response format is a JSON string, rather than a tag. However, for clarity and ease of presentation, we have used tags in the figure to represent the output.

Reflection

Question: *Where was the place of burial of Djedefhor's father?*
Ground Truth: *Great Pyramid of Giza*

Response Trajectory from DeepResearcher:
<think>
To answer this question, I need to find out information about Djedefhor's father and the place of his burial. I will start with a web search.
</think>
<search>
Herschell Gordon Lewis filmmaker pioneer
Leah Meyerhoff filmmaker pioneer
place of burial of Djedefhor's father
Djedefhor father burial site
who was Djedefhor's father and where was he buried
</search>
<tool_response>
...(omitted to save space)
</tool_response>
<think>
The web search results provide clues about Djedefhor's father. **However, the search results are mostly about Boromir's father Denethor from the Lord of the Rings, not Djedefhor's father. I need to refine my search to find information about Djedefhor specifically.**
</think>
<search>
who was Djedefhor's father and where was he buried
Djedefhor father name
Djedefhor father identity
</search>
<tool_response>
...(omitted to save space)
</tool_response>
<think>
The new search results provide information about Djedefhor's father, Khufu, and also his burial place. **However, the search results for his burial need to be more specific.** I will use the web browser tool to look at the pages where the information may be more relevant.
</think>
<browse_webpage>
https://en.wikipedia.org/wiki/Djedefre
https://www.ancient-egypt.org/history/old-kingdom/4th-dynasty/djedefre/biography-of-djedefre.html
https://ancientegyptonline.co.uk/djedefre/
</browse_webpage>
<tool_response>
...(omitted to save space)
</tool_response>
<think>
From the browsing pages, it is evident that Djedefhor's father is Khufu. The pages specifically mention that Khufu was the father of Djedefhor. However, I need to find the place of burial of Djedefhor's father, Khufu.
</think>
<search>
place of burial of Khufu
Khufu burial site
where was Khufu buried
</search>
<tool_response>
...(omitted to save space)
</tool_response>
<think>
The search results clearly indicate that Khufu's burial place was in the Great Pyramid of Giza. This pyramid served as the tomb for pharaoh Khufu.
</think>
<answer>
Great Pyramid of Giza
</answer>

Reflection and adjust research direction

Honesty

Question: *In the seventh-largest country in Africa, what is the oil prodcution of the city at the mouth of the Congo River?*
Ground Truth: *1200000 oilbbl/d*

Response Trajectory from DeepResearcher:
<think>
To answer this question, I need to identify the city at the mouth of the Congo River in the seventh-largest country in Africa, and then find its current oil production.
</think>
<search>
seventh largest country in africa
city at the mouth of the Congo River in the seventh largest country in africa
</search>
<tool_response>
...(omitted to save space)
</tool_response>
<think>
From the search results, I found that Angola is the seventh largest country in Africa, and its capital Luanda is located near the mouth of the Congo River. Now I need to find the oil production of Luanda.
</think>
<search>
oil production of Luanda
oil production in Angola
</search>
<tool_response>
...(omitted to save space)
</tool_response>
<think>
From the search results, I can see that Angola currently produces around 1.16 million barrels of oil per day. However, I should find the specific production of the city at the mouth of the Congo River, which is Luanda.
</think>
<search>
oil production of Luanda
oil production in Luanda
</search>
<tool_response>
...(omitted to save space)
</tool_response>
<think>
After a series of web searches, I couldn't find specific information about the oil production of Luanda. The results are mostly about Angola's total oil production. **Since Luanda is the capital and the largest city of Angola, it's likely that a significant portion of Angola's oil production happens there.** For a more exact figure, we might need to explore specialized databases or recent reports about the oil industry in Luanda.
</think>
<browse_webpage>
https://en.wikipedia.org/wiki/Luanda
https://www.worldometers.info/country-data/angola/oil-production
</browse_webpage>
<think>
It seems the information about the oil production of Luanda is not directly available in the search results or in the previously browsed webpages. Given the information from previous searches, Angola is a major oil producer, and as the capital and largest city, Luanda likely plays a key role in the country's oil production. However, to answer the question precisely, we may need a more detailed report on the city's oil production, which might be included in specialized databases or recent reports. **Since the exact oil production of Luanda is not available from the searches and browsing**, and given the context and the previously found information that Angola is a major oil producer, **it's reasonable to infer that the oil production in the city at the mouth of the Congo River (Luanda) is a significant part** of Angola's total oil production. **However, without a precise figure, I will not provide a number.**
</think>
<answer>
significant portion
</answer>

Be honest when didn't find the exact answer

Figure 4: Reflection and Honesty Behavior. The search and browse are 2 apis in json format in the real inference stage. Left: When the retrieved information does not fully align with the question, DeepResearcher recognizes this discrepancy based on environmental feedback and refines its search query in subsequent tool calls. This proves its reflection ability. Right: DeepResearcher is capable of recognizing when it has not found the correct answer and appropriately declines to provide a response to be honesty. Note: In the actual model, the tool call and tool response format is a JSON string, rather than a tag. However, for clarity and ease of presentation, we have used tags in the figure to represent the output.

C Prompts

C.1 Prompt for Question Quality Level Evaluation

The prompt below displays two templates. Identifies if questions are time-sensitive, subjective, or potentially harmful. Includes classification guidelines, question placeholder, and required answer tag format.

Prompt for training data quality checking

Please identify whether the given question is time-sensitive, subjective, or may cause harmful answers.

- Time-sensitive: The answer to the question may change over time.
- Harmful: The answer to the question may be harmful or offensive.
- Subjective: The answer to the question may be subjective and not based on facts.

Here is the question:
<question>
{question}
</question>

Wrap your answer in <answer> tags with one of the following values:
- time_sensitive: if the question is time-sensitive
- harmful: if the question may cause harmful answers
- subjective: if the question is subjective
- good: if the question is none of the above

The prompt below shows the template prompt for contamination detection. To tests if AI responses are influenced by training data contamination.

Prompt for contamination detection

Give a short answer to the following question. The answer should be in English.

Question: {question}

Your answer:

C.2 Prompt for Model's Answer Quality Level Evaluation

The prompt below provides instructions for evaluating the correctness of AI-generated answers (pred answer) against a list of ground truth answers. To judge if a predicted answer correctly answers a question by comparing it to ground truth answers.

Prompt for Model-based Evaluation

You will be given a question and its ground truth answer list where each item can be a ground truth answer. Provided a pred_answer, you need to judge if the pred_answer correctly answers the question based on the ground truth answer list.
You should first give your rationale for the judgement, and then give your judgement result (i.e., correct or incorrect).

Here is the criteria for the judgement:
1. The pred_answer doesn't need to be exactly the same as any of the ground truth answers, but should be semantically same for the question.
2. Each item in the ground truth answer list can be viewed as a ground truth answer for the question, and the pred_answer should be semantically same to at least one of them.

question: {question}
ground truth answers: {gt_answer}
pred_answer: {pred_answer}

The output should in the following json format:
'''json
{
"rationale": "your rationale for the judgement, as a text",
"judgement": "your judgement result, can only be 'correct' or 'incorrect'"
}
'''

Your output:

C.3 Prompt for Research Plan on Question Answering

The prompt below outlines the structured approach for addressing complex questions, utilizing web search and webpage browsing tools to conduct in-depth research and gather the necessary information for a comprehensive response.

Prompt for Research Plan on Complex Question Answering

Background information
* Today is YYYY-MM-DD
* You are Deep AI Research Assistant
The question I give you is a complex question that requires a *deep research* to answer.

I will provide you with two tools to help you answer the question:
* A web search tool to help you perform google search.
* A webpage browsing tool to help you get new page content.

You don't have to answer the question now, but you should first think about the research plan or what to search next.

Your output format should be one of the following two formats:

<think>
YOUR THINKING PROCESS
</think>
<answer>
YOUR ANSWER AFTER GETTING ENOUGH INFORMATION
</answer>
or
<think>
YOUR THINKING PROCESS
</think>
<tool_call>
YOUR TOOL CALL WITH CORRECT FORMAT
</tool_call>

You should always follow the above two formats strictly.
Only output the final answer (in words, numbers or phrase) inside the <answer></answer> tag, without any explanations or extra information. If this is a yes-or-no question, you should only answer yes or no.

Tools
You may call one or more functions to assist with the user query.
You are provided with function signatures within <tools></tools> XML tags:
<tools>
{'type': 'function', 'function': {'name': 'web_search', 'description': 'Search the web for relevant information from google. You should use this tool if the historical page content is not enough to answer the question. Or last search result is not relevant to the question.', 'parameters': {'type': 'object', 'properties': {'query': {'type': 'array', 'items': {'type': 'string', 'description': 'The query to search, which helps answer the question'}, 'description': 'The queries to search'}}, 'required': ['query'], 'minItems': 1, 'uniqueItems': true}}}
{'type': 'function', 'function': {'name': 'browse_webpage', 'description': 'Browse the webpage and return the content that not appeared in the conversation history. You should use this tool if the last action is search and the search result maybe relevant to the question.', 'parameters': {'type': 'object', 'properties': {'url_list': {'type': 'array', 'items': {'type': 'string', 'description': 'The chosen url from the search result, do not use url that not appeared in the search result'}, 'description': 'The chosen urls from the search result.'}}, 'required': ['url_list']}}}
</tools>

For each function call, return a json object with function name and arguments within <tool_call></tool_call> XML tags:

<tool_call>

"name": <function-name>, "arguments": <args-json-object>

</tool_call>

D Training Scaling Result

Figure 5 presents the F1 score in 7 benchmarks. We sampled 125 cases from each benchmarks' development set. DeepResearcher can scale in all benchmarks, especially in OOD benchmarks.

E Performance

Figure 6 provides a consolidated visualization of DeepResearcher's performance in comparison to other models across a comprehensive suite of seven distinct datasets. This consistent outperformance not only serves as a robust validation of the model's advanced capabilities and effectiveness but also strongly indicates its significant generalization ability across diverse data domains and task types.

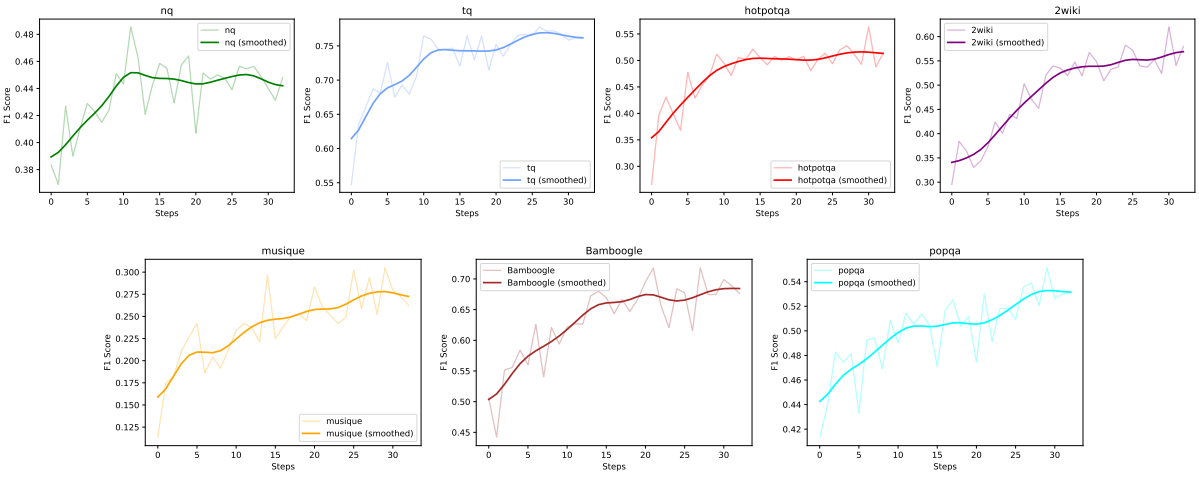


Figure 5: F1 score during training

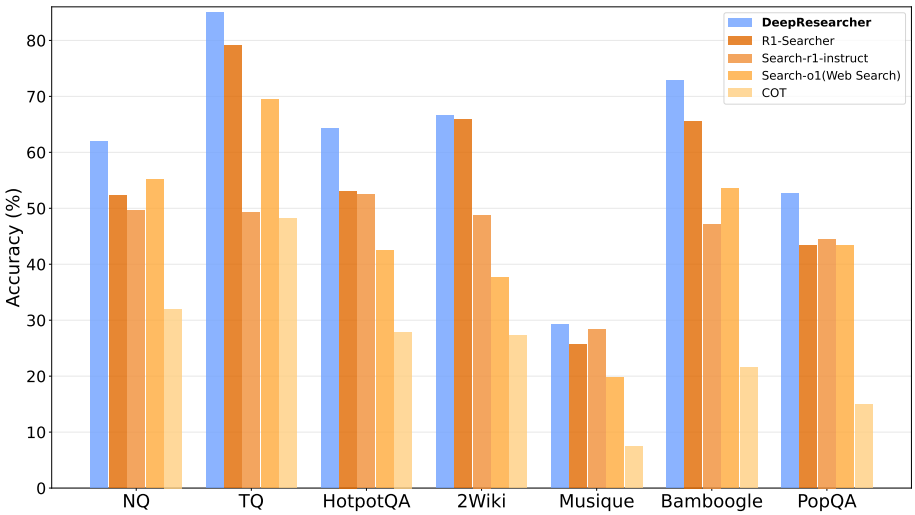


Figure 6: DeepResearcher performs the best on all 7 datasets measured by reliable model-based evaluation.